

Domania Backend Starter — Node.js + Express + PostgreSQL + Cloud Run

1. Structure du projet

```
domania-backend/  
├── src/  
│   ├── config/  
│   │   └── db.js  
│   ├── middleware/  
│   │   └── auth.js  
│   ├── routes/  
│   │   ├── auth.routes.js  
│   │   ├── properties.routes.js  
│   │   ├── contracts.routes.js  
│   │   └── health.routes.js  
│   ├── server.js  
│   └── schema.sql  
├── .env.example  
├── Dockerfile  
├── package.json  
└── README.md
```

2. package.json

```
{  
  "name": "domania-backend",  
  "version": "1.0.0",  
  "description": "Backend API for CitySoft Domania SaaS",  
  "main": "src/server.js",  
  "type": "module",  
  "scripts": {  
    "start": "node src/server.js",  
    "dev": "nodemon src/server.js"  
  },  
  "dependencies": {  
    "bcryptjs": "^2.4.3",  
    "cors": "^2.8.5",  
    "dotenv": "^16.4.5",  
    "express": "^4.19.2",  
  }  
}
```

```
    "helmet": "^7.1.0",
    "jsonwebtoken": "^9.0.2",
    "pg": "^8.12.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.4"
  }
}
```

3. .env.example

```
PORT=8080
NODE_ENV=development
DATABASE_URL=postgresql://postgres:password@localhost:5432/domania
JWT_SECRET=change_this_secret_key
CORS_ORIGIN=http://localhost:3000
```

4. Dockerfile

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --omit=dev

COPY . .

ENV PORT=8080
EXPOSE 8080

CMD ["npm", "start"]
```

5. src/config/db.js

```
import pg from "pg";
import dotenv from "dotenv";
```

```

dotenv.config();

const { Pool } = pg;

export const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: process.env.NODE_ENV === "production" ? { rejectUnauthorized: false } :
  false,
});

export async function query(text, params) {
  const start = Date.now();
  const result = await pool.query(text, params);
  const duration = Date.now() - start;
  console.log("Executed query", { text, duration, rows: result.rowCount });
  return result;
}

```

6. src/middleware/auth.js

```

import jwt from "jsonwebtoken";

export function requireAuth(req, res, next) {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Token manquant ou invalide" });
  }

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (error) {
    return res.status(401).json({ message: "Session expirée ou token
invalide" });
  }
}

```

7. src/routes/health.routes.js

```
import express from "express";

const router = express.Router();

router.get("/", (req, res) => {
  res.json({
    status: "ok",
    service: "domania-backend",
    timestamp: new Date().toISOString(),
  });
});

export default router;
```

8. src/routes/auth.routes.js

```
import express from "express";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import { query } from "../config/db.js";

const router = express.Router();

router.post("/register", async (req, res) => {
  try {
    const { full_name, email, password, role = "admin", tenant_name } =
      req.body;

    if (!full_name || !email || !password || !tenant_name) {
      return res.status(400).json({ message: "Champs obligatoires manquants" });
    }

    const existingUser = await query("SELECT id FROM users WHERE email = $1",
[email]);
    if (existingUser.rows.length > 0) {
      return res.status(409).json({ message: "Cet email existe déjà" });
    }

    const tenantResult = await query(
      "INSERT INTO tenants (name) VALUES ($1) RETURNING id, name",
      [tenant_name]
```

```

    );

    const tenant = tenantResult.rows[0];
    const hashedPassword = await bcrypt.hash(password, 10);

    const userResult = await query(
      `INSERT INTO users (tenant_id, full_name, email, password_hash, role)
      VALUES ($1, $2, $3, $4, $5)
      RETURNING id, tenant_id, full_name, email, role, created_at`,
      [tenant.id, full_name, email, hashedPassword, role]
    );

    const user = userResult.rows[0];

    const token = jwt.sign(
      { id: user.id, tenant_id: user.tenant_id, role: user.role, email:
user.email },
      process.env.JWT_SECRET,
      { expiresIn: "7d" }
    );

    res.status(201).json({ user, tenant, token });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Erreur serveur" });
  }
});

router.post("/login", async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ message: "Email et mot de passe requis" });
    }

    const result = await query("SELECT * FROM users WHERE email = $1", [email]);
    const user = result.rows[0];

    if (!user) {
      return res.status(401).json({ message: "Identifiants incorrects" });
    }

    const isValid = await bcrypt.compare(password, user.password_hash);
    if (!isValid) {
      return res.status(401).json({ message: "Identifiants incorrects" });
    }
  }
});

```

```

    const token = jwt.sign(
      { id: user.id, tenant_id: user.tenant_id, role: user.role, email:
user.email },
      process.env.JWT_SECRET,
      { expiresIn: "7d" }
    );

    delete user.password_hash;
    res.json({ user, token });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Erreur serveur" });
  }
});

export default router;

```

9. src/routes/properties.routes.js

```

import express from "express";
import { query } from "../config/db.js";
import { requireAuth } from "../middleware/auth.js";

const router = express.Router();

router.use(requireAuth);

router.get("/", async (req, res) => {
  try {
    const result = await query(
      "SELECT * FROM properties WHERE tenant_id = $1 ORDER BY created_at DESC",
      [req.user.tenant_id]
    );
    res.json(result.rows);
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Erreur serveur" });
  }
});

router.post("/", async (req, res) => {
  try {
    const { title, type, address, city, monthly_rent, status = "available" } =
req.body;

```

```

    const result = await query(
      `INSERT INTO properties (tenant_id, title, type, address, city,
monthly_rent, status)
      VALUES ($1, $2, $3, $4, $5, $6, $7)
      RETURNING *`,
      [req.user.tenant_id, title, type, address, city, monthly_rent, status]
    );

    res.status(201).json(result.rows[0]);
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Erreur serveur" });
  }
});

router.get("/:id", async (req, res) => {
  try {
    const result = await query(
      "SELECT * FROM properties WHERE id = $1 AND tenant_id = $2",
      [req.params.id, req.user.tenant_id]
    );

    if (result.rows.length === 0) {
      return res.status(404).json({ message: "Bien introuvable" });
    }

    res.json(result.rows[0]);
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Erreur serveur" });
  }
});

export default router;

```

10. src/routes/contracts.routes.js

```

import express from "express";
import { query } from "../config/db.js";
import { requireAuth } from "../middleware/auth.js";

const router = express.Router();

```

```

router.use(requireAuth);

router.get("/", async (req, res) => {
  try {
    const result = await query(
      `SELECT c.*, p.title AS property_title
      FROM contracts c
      JOIN properties p ON p.id = c.property_id
      WHERE c.tenant_id = $1
      ORDER BY c.created_at DESC`,
      [req.user.tenant_id]
    );

    res.json(result.rows);
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Erreur serveur" });
  }
});

router.post("/", async (req, res) => {
  try {
    const {
      property_id,
      tenant_full_name,
      tenant_phone,
      start_date,
      end_date,
      monthly_rent,
      deposit_amount,
      status = "draft",
    } = req.body;

    const result = await query(
      `INSERT INTO contracts
      (tenant_id, property_id, tenant_full_name, tenant_phone, start_date,
end_date, monthly_rent, deposit_amount, status)
VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)
RETURNING *`,
      [
        req.user.tenant_id,
        property_id,
        tenant_full_name,
        tenant_phone,
        start_date,
        end_date,
        monthly_rent,
        deposit_amount,
        status,
      ]
    );
  }
});

```

```

        status,
    ]
    });

    res.status(201).json(result.rows[0]);
} catch (error) {
    console.error(error);
    res.status(500).json({ message: "Erreur serveur" });
}
});

export default router;

```

11. src/server.js

```

import express from "express";
import cors from "cors";
import helmet from "helmet";
import dotenv from "dotenv";

import healthRoutes from "./routes/health.routes.js";
import authRoutes from "./routes/auth.routes.js";
import propertiesRoutes from "./routes/properties.routes.js";
import contractsRoutes from "./routes/contracts.routes.js";

dotenv.config();

const app = express();
const PORT = process.env.PORT || 8080;

app.use(helmet());
app.use(express.json({ limit: "10mb" }));
app.use(
    cors({
        origin: process.env.CORS_ORIGIN || "*",
        credentials: true,
    })
);

app.get("/", (req, res) => {
    res.json({
        message: "Bienvenue sur l'API Domania",
        version: "1.0.0",
    });
});

```

```

});

app.use("/health", healthRoutes);
app.use("/api/auth", authRoutes);
app.use("/api/properties", propertiesRoutes);
app.use("/api/contracts", contractsRoutes);

app.use((req, res) => {
  res.status(404).json({ message: "Route introuvable" });
});

app.listen(PORT, () => {
  console.log(`Domania backend running on port ${PORT}`);
});

```

12. src/schema.sql

```

CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TABLE IF NOT EXISTS tenants (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(255) NOT NULL,
  plan VARCHAR(50) DEFAULT 'starter',
  status VARCHAR(50) DEFAULT 'active',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID REFERENCES tenants(id) ON DELETE CASCADE,
  full_name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  role VARCHAR(50) DEFAULT 'admin',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS properties (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id UUID REFERENCES tenants(id) ON DELETE CASCADE,
  title VARCHAR(255) NOT NULL,
  type VARCHAR(100),
  address TEXT,
  city VARCHAR(150),

```

```

    monthly_rent NUMERIC(12, 2),
    status VARCHAR(50) DEFAULT 'available',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS contracts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID REFERENCES tenants(id) ON DELETE CASCADE,
    property_id UUID REFERENCES properties(id) ON DELETE CASCADE,
    tenant_full_name VARCHAR(255) NOT NULL,
    tenant_phone VARCHAR(100),
    start_date DATE,
    end_date DATE,
    monthly_rent NUMERIC(12, 2),
    deposit_amount NUMERIC(12, 2),
    status VARCHAR(50) DEFAULT 'draft',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_users_tenant_id ON users(tenant_id);
CREATE INDEX IF NOT EXISTS idx_properties_tenant_id ON properties(tenant_id);
CREATE INDEX IF NOT EXISTS idx_contracts_tenant_id ON contracts(tenant_id);
CREATE INDEX IF NOT EXISTS idx_contracts_property_id ON contracts(property_id);

```

13. README.md

```

# Domania Backend

Backend API for CitySoft Domania SaaS.

## Installation locale

```bash

npm install
cp .env.example .env
npm run dev

```

## Variables d'environnement

```

PORT=8080
DATABASE_URL=postgres://postgres:password@localhost:5432/domania

```

```
JWT_SECRET=change_this_secret_key
CORS_ORIGIN=http://localhost:3000
```

## Routes principales

### Health

GET /health

### Auth

POST /api/auth/register POST /api/auth/login

### Properties

GET /api/properties POST /api/properties GET /api/properties/:id

### Contracts

GET /api/contracts POST /api/contracts

## Déploiement Cloud Run

Cloud Run utilise automatiquement le port 8080.

Pour le déploiement depuis GitHub : 1. Connecter le dépôt GitHub 2. Choisir Cloud Run 3. Région : europe-west1 4. Autoriser accès non authentifié 5. Ajouter les variables d'environnement

```

14. Test rapide après déploiement

Une fois Cloud Run déployé :

```bash
curl https://TON-URL-CLOUD-RUN.run.app/health
```

Réponse attendue :

```
{
  "status": "ok",
```

```
"service": "domania-backend"  
}
```